



UNIVERSITÀ DELLA CALABRIA
DIPARTIMENTO DI INGEGNERIA INFORMATICA,
MODELLISTICA, ELETTRONICA E SISTEMISTICA

Terzo progetto:

Sommatore a 6 operandi ad 8 bit in complemento a 2

Studenti:

Thomas Michienzi

Mat. 250780

Pasquale Morabito

Mat. 250633

Emanuele Nisticò

Mat. 251218

Professoressa del corso:

Stefania Perri

ANNO ACCADEMICO 2025/2026

Indice

1	Descrizione del circuito	2
2	Descrizione VHDL	4
2.1	Simulazione logica	9
3	Sintesi	11
3.1	Simulazione post-sintesi	14
4	Implementazione	15
4.1	Simulazione post-implementazione	16
5	Massimizzazione della frequenza di funzionamento	17

1 Descrizione del circuito

L'obiettivo del progetto è realizzare un circuito che, dati 6 ingressi a 8 bit in complemento a 2, ne effettua la somma impiegando almeno una pipeline. Una prima possibilità sarebbe stata utilizzare una pipeline costituita da sommatore a due ingressi, ma in questo scenario la latenza (ossia il tempo necessario a far riempire completamente la pipe), sarebbe risultata eccessiva. Essa viene infatti definita come:

$$\text{Latenza} = n - 1 \quad \text{con } n := n^\circ \text{ di regitri lungo un percorso}$$

Da cui segue che $L = 6 - 1 = 5$. Dunque, per ottimizzare i tempi di lavoro del circuito, in fase progettuale è stato realizzato un sommatore a tre operandi dotato di una struttura di tipo pipeline. Tale sommatore è stato poi istanziato due volte, ciascuna per elaborare un gruppo di tre operandi. Questa scelta consente l'esecuzione parallela delle operazioni di somma, permettendo di ottenere simultaneamente i risultati dei due blocchi. Di seguito riportiamo lo schema del sommatore a tre ingressi:

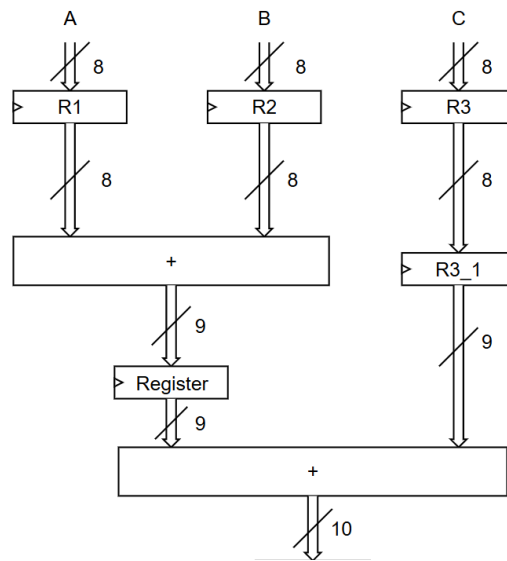


Figura 1: Sommatore a 3 ingressi in struttura pipeline

Al primo livello della pipeline, troviamo un insieme di registri, il cui numero è pari a quello degli ingressi (tre in questo caso). Questi sono sensibili al fronte di salita e sono fondamentali per la sincronizzazione dei segnali: la loro presenza garantisce che tutti i dati in ingresso vengano campionati e allineati temporalmente esattamente al primo fronte di salita del segnale di clock. Il secondo livello ospita il primo stadio di calcolo, costituito da un sommatore che esegue la somma parziale tra i primi due dei tre ingressi precedentemente registrati.

Immediatamente dopo la prima somma si colloca il terzo livello. Questo è composto da due registri distinti: un registro cattura e memorizza il risultato della somma parziale appena eseguita, mentre l'altro registro è dedicato all'ingresso che non è stato ancora sommato. La funzione cruciale di questo secondo registro è il bilanciamento dei rami, infatti assicura che il ritardo di propagazione dell'ingresso non sommato sia perfettamente allineato a quello della somma parziale, permettendo così ai due segnali di raggiungere simultaneamente il prossimo livello.

Il rispettivo risultato dei due sommatore a 3 ingressi, sono messi in ingresso ad un ulteriore sommatore che produce il risultato finale. Tale struttura, detta *ad albero* è descritta dalla seguente figura:

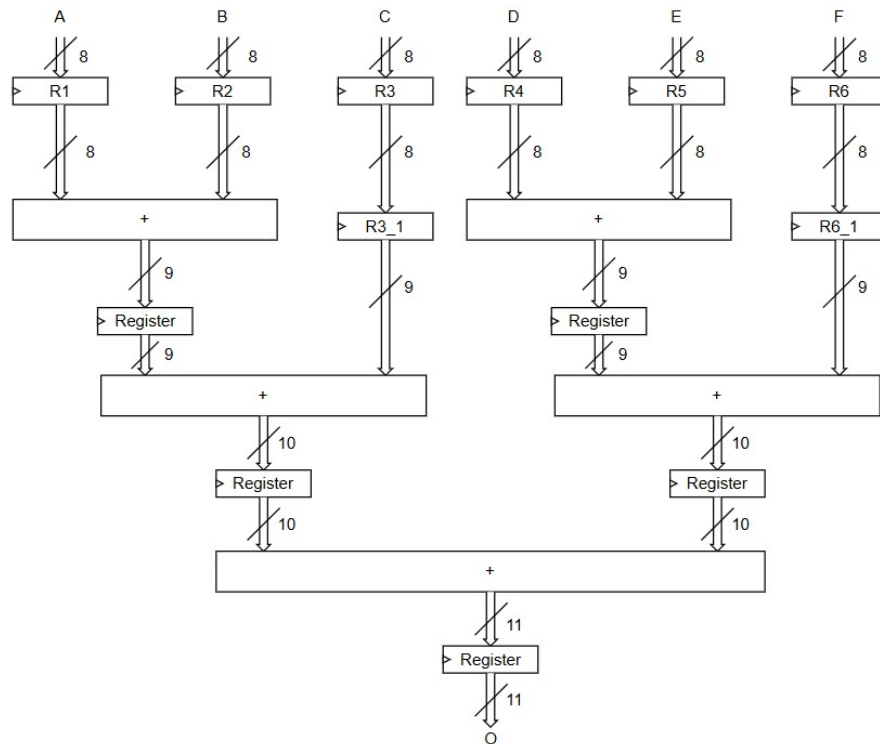


Figura 2: Schema circuito

Utilizzando questo approccio il primo risultato arriverà dopo 3 colpi di clock, dal momento che lungo un percorso troviamo 4 registri. Dopo il primo risultato, il circuito inizierà a lavorare a regime, infatti ad ogni colpo di clock otterremo un risultato.

2 Descrizione VHDL

Il circuito graficamente descritto nel precedente paragrafo è stato rappresentato attraverso il linguaggio VHDL con un approccio di tipo strutturale. Andremo ad analizzare i vari componenti realizzati a questo fine con un approccio di tipo bottom up.

Per una facilità nella scrittura dei codici successivi si è deciso di andare a creare un componente rappresentante un registro costituito da una serie di flip-flop sensibili ai fronti salita di dimensione generica. Questo rappresenterà l'elemento con cui otterremo la sincronizzazione del circuito.

Listing 1: File RegisterFF.vhd

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity RegisterFF is
5     generic(n : integer := 8);
6     Port ( D : in STD_LOGIC_VECTOR (n - 1 downto 0);
7           clk : in STD_LOGIC;
8           Q : out STD_LOGIC_VECTOR (n - 1 downto 0));
9 end RegisterFF;
10
11 architecture Behavioral of RegisterFF is
12
13 begin
14     process (clk) begin
15         if rising_edge(clk) then
16             Q <= D;
17         end if;
18     end process;
19 end Behavioral;
```

Il componente centrale del sommatore a sei operandi è un sommatore a due operandi, anch'esso a dimensione generica. Come scelta implementativa, si è deciso di integrare direttamente all'interno di questo modulo il registro che memorizza il risultato finale dell'operazione, necessario per lo schema operativo adottato.

Listing 2: File AdderRegister.vhd

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity AdderRegister is
5     generic (n : integer := 8);
6     Port ( A : in STD_LOGIC_VECTOR (n - 1 downto 0);
7           B : in STD_LOGIC_VECTOR (n - 1 downto 0);
8           clk : in STD_LOGIC;
9           Q : out STD_LOGIC_VECTOR(n downto 0));
10 end AdderRegister;
```

```

11
12 architecture Behavioral of AdderRegister is
13     signal EA, EB: STD_LOGIC_VECTOR(n downto 0);
14     signal p, g: STD_LOGIC_VECTOR(n downto 0);
15     signal c: STD_LOGIC_VECTOR(n + 1 downto 0);
16     signal S: STD_LOGIC_VECTOR(n downto 0);      --
17         uscita del sommatore
18 begin
19     -- calcolo combinatorio
20     -- estensione in segno
21     EA <= A(n - 1) & A;
22     EB <= B(n - 1) & B;
23
24     p <= EA xor EB;
25     g <= EA and EB;
26     c(0) <= '0';
27     c(n + 1 downto 1) <= g or (p and c(n downto 0));
28
29     -- calcolo del risultato
30     S <= p xor c(n downto 0);
31
32     -- memorizzazione del risultato
33     -- registro a flip-flop sensibile al fronte di salita
34     process (clk) begin
35         if rising_edge(clk) then
36             Q <= S;
37         end if;
38     end process;
39 end Behavioral;

```

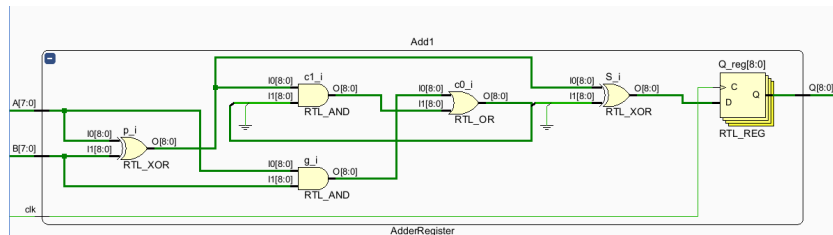


Figura 3: Schematic dell'AdderRegister ad 8 bit

A questo punto arrivati, è molto semplice definire il sommatore a tre operandi con pipeline , analizzato nel precedente paragrafo , come una combinazione dei precedenti moduli.

Listing 3: File Pipeline.vhd

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;

```

```

3
4 entity Pipeline is
5     Port (
6         A : in STD_LOGIC_VECTOR(7 downto 0);
7         B : in STD_LOGIC_VECTOR(7 downto 0);
8         C : in STD_LOGIC_VECTOR(7 downto 0);
9         O : out STD_LOGIC_VECTOR(9 downto 0);
10        clk : in STD_LOGIC
11    );
12 end Pipeline;
13
14 architecture Behavioral of Pipeline is
15     component AdderRegister
16         generic (n : integer := 8);
17         Port( A : in STD_LOGIC_VECTOR (n - 1 downto 0);
18              B : in STD_LOGIC_VECTOR (n - 1 downto 0);
19              clk : in STD_LOGIC;
20              Q : out STD_LOGIC_VECTOR(n downto 0)
21        );
22     end component;
23     component RegisterFF
24         generic(n : integer := 8);
25         Port ( D : in STD_LOGIC_VECTOR (n - 1 downto 0);
26              clk : in STD_LOGIC;
27              Q : out STD_LOGIC_VECTOR (n - 1 downto 0)
28        );
29     end component;
30     signal S1, S2, S3, S3_1: STD_LOGIC_VECTOR(7 downto 0);
31     signal S_A: STD_LOGIC_VECTOR(8 downto 0);
32     signal ES3_1 : STD_LOGIC_VECTOR(8 downto 0);
33     begin
34         -- Primo livello
35         R1 : RegisterFF generic map(8) port map (A, clk, S1)
36             ;
37         R2 : RegisterFF generic map(8) port map (B, clk, S2)
38             ;
39         R3 : RegisterFF generic map(8) port map (C, clk, S3)
40             ;
41
42         Add1 : AdderRegister generic map (8) port map (S1,
43             S2, clk, S_A);
44         R3_1 : RegisterFF generic map (8) port map (S3, clk,
45             S3_1);
46         ES3_1 <= S3_1(7) & S3_1;
47
48         -- Secondo livello
49         Add2: AdderRegister generic map (9) port map (S_A,
50             ES3_1, clk, O);
51     end Behavioral;

```

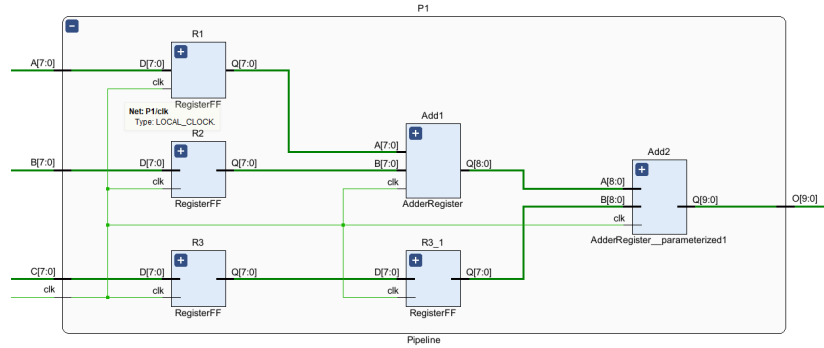


Figura 4: Schematic del sommatore a 3 operandi

Concludendo, il nostro circuito è costituito da due componenti Pipeline (ciascuno dei quali prende una partizione degli operandi in ingresso), i cui risultati vengono sommati dall'AdderRegister.

Listing 4: File BigAdder.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity BigAdder is
5      generic (n : integer := 8);
6      Port (
7          A : in STD_LOGIC_VECTOR (7 downto 0);
8          B : in STD_LOGIC_VECTOR (7 downto 0);
9          C : in STD_LOGIC_VECTOR (7 downto 0);
10         D : in STD_LOGIC_VECTOR (7 downto 0);
11         E : in STD_LOGIC_VECTOR (7 downto 0);
12         F : in STD_LOGIC_VECTOR (7 downto 0);
13         O : out STD_LOGIC_VECTOR (10 downto 0);
14         clk : in STD_LOGIC
15     );
16  end BigAdder;
17
18  architecture Behavioral of BigAdder is
19      component AdderRegister
20          generic (n : integer := 8);
21          Port (
22              A : in STD_LOGIC_VECTOR(n - 1 downto 0);
23              B : in STD_LOGIC_VECTOR(n - 1 downto 0);
24              clk : in STD_LOGIC;
25              Q : out STD_LOGIC_VECTOR(n downto 0)
26          );

```



```

27   end component;
28   component Pipeline
29     Port (
30       A : in STD_LOGIC_VECTOR(7 downto 0);
31       B : in STD_LOGIC_VECTOR(7 downto 0);
32       C : in STD_LOGIC_VECTOR(7 downto 0);
33       O : out STD_LOGIC_VECTOR(9 downto 0);
34       clk : in STD_LOGIC
35     );
36   end component;
37   signal S1, S2 : STD_LOGIC_VECTOR(9 downto 0);
38   begin
39     -- Primo livello
40     P1 : Pipeline port map (A, B, C, S1, clk);
41     P2 : Pipeline port map (D, E, F, S2, clk);
42     --Secondo livello
43     Add : AdderRegister generic map (10) port map (S1,
44       S2, clk, 0);
45   end Behavioral;

```

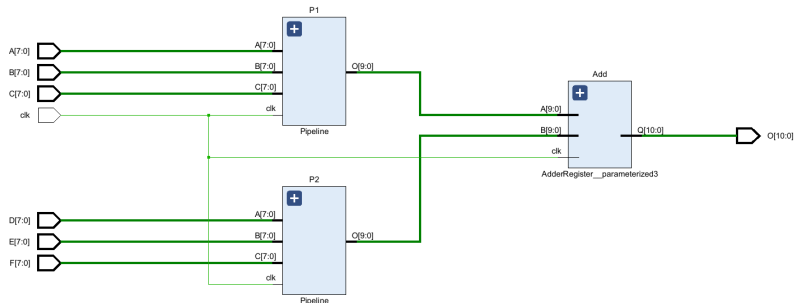


Figura 5: Schematic del sommatore a 6 operandi

Il circuito complessivo ottenuto è il seguente.

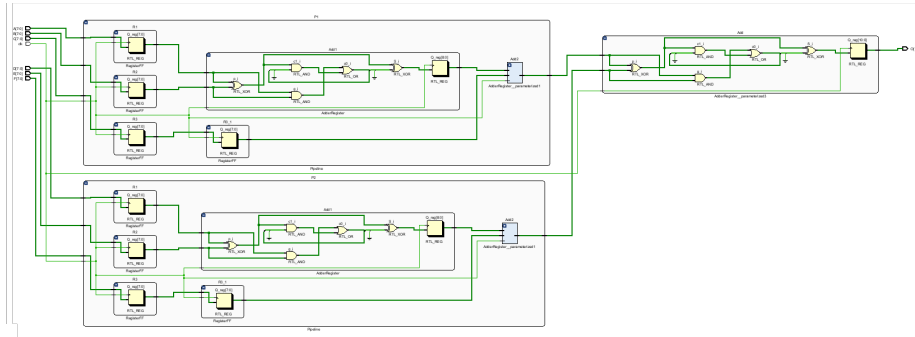


Figura 6: Schematic "esteso" del sommatore a 6 operandi

2.1 Simulazione logica

Per verificare la correttezza della logica implementata, eseguiamo la simulazione logica del circuito sviluppando un file di testbench. Dato il numero degli ingressi, risulta onerosa la simulazione esaustiva per tutti i loro possibili valori. Proprio per questo abbiamo evidenziato 20 casi di test:

Listing 5: File TestBench.vhdl

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4
5  entity TestBench is
6  -- Port ( );
7  end TestBench;
8
9  architecture Behavioral of TestBench is
10     component BigAdder
11         Port (
12             A : in STD_LOGIC_VECTOR (7 downto 0);
13             B : in STD_LOGIC_VECTOR (7 downto 0);
14             C : in STD_LOGIC_VECTOR (7 downto 0);
15             D : in STD_LOGIC_VECTOR (7 downto 0);
16             E : in STD_LOGIC_VECTOR (7 downto 0);
17             F : in STD_LOGIC_VECTOR (7 downto 0);
18             O : out STD_LOGIC_VECTOR (10 downto 0);
19             clk : in STD_LOGIC
20         );
21     end component;
22     signal A, B, C, D, E, F : STD_LOGIC_VECTOR(7 downto 0);
23     signal clk : STD_LOGIC;
24     signal O : STD_LOGIC_VECTOR(10 downto 0);
25     constant T : time := 10 ns;          -- semiperiodo di clock

```

```

26 begin
27     -- clock (periodo := 20ns, duty cycle := 50%)
28     process begin
29         clk <= '0';
30         wait for T;
31         clk <= '1';
32         wait for T;
33     end process;
34
35     -- ingressi di test
36     process begin
37         for i in -10 to 9 loop
38             A <= conv_std_logic_vector(i, 8);
39             E <= conv_std_logic_vector(127 - i, 8);
40             B <= conv_std_logic_vector(-128 + i, 8);
41             C <= conv_std_logic_vector(63 + i, 8);
42             D <= conv_std_logic_vector(-32 - i, 8);
43             F <= conv_std_logic_vector(-16 + i, 8);
44             wait until falling_edge(clk);
45         end loop;
46     end process;
47
48     CUT: BigAdder port map(A, B, C, D, E, F, O, clk);
49 end Behavioral;

```

Dall'esito della simulazione mostrato nella figura seguente si può notare che l'uscita del circuito (il segnale *O*) rimane indefinita (nello standard implementato dal tipo STD_LOGIC questo stato è rappresentato dalla lettera *U*). Questo è dovuto al fatto che la prima uscita viene prodotta dal circuito al quarto colpo di clock e, fino a quel momento, il suo stato rimane indeterminato. Sottolineiamo infine che il fatto che l'uscita venga prodotta al quarto colpo di clock rispetta il calcolo sulla latenza fatto nella sezione precedente.

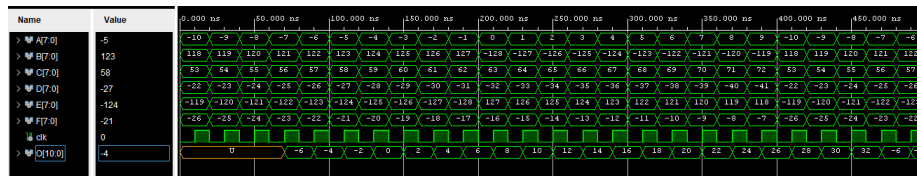


Figura 7: Simulazione ideale del sommatore a 6 operandi

3 Sintesi

Eseguiamo adesso la sintesi del circuito secondo la logica della tecnologia FPGA. Per questioni di praticità, mostriamo il risultato ottenuto in più immagini:

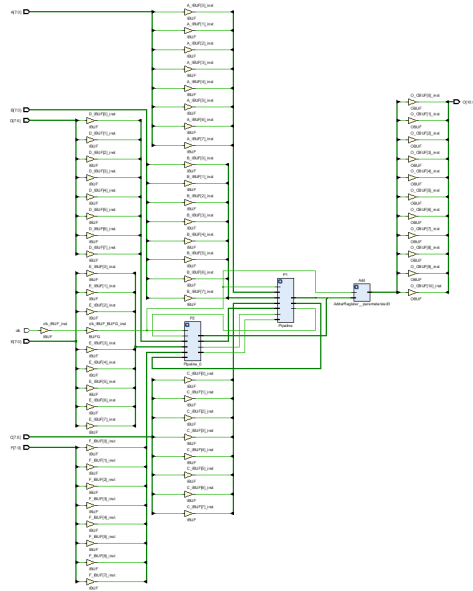


Figura 8: Schema complessivo del circuito sintetizzato

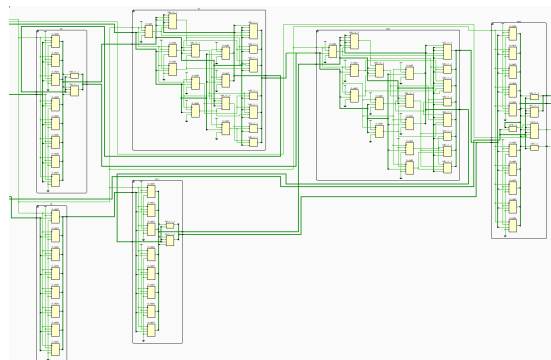


Figura 9: Particolare della pipeline sintetizzata

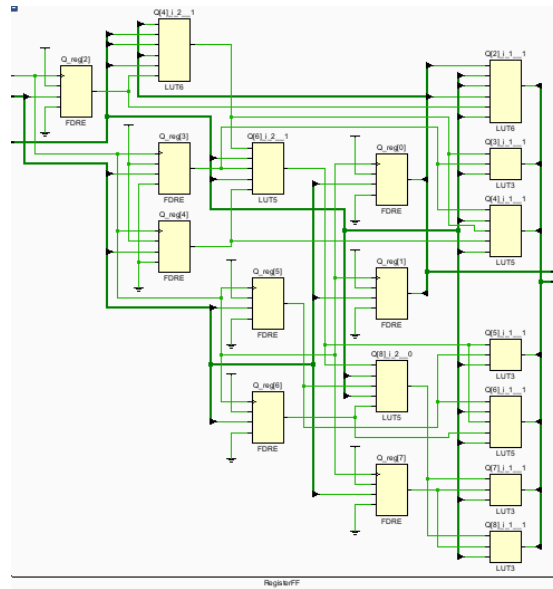


Figura 10: Particolare del registro sintetizzato

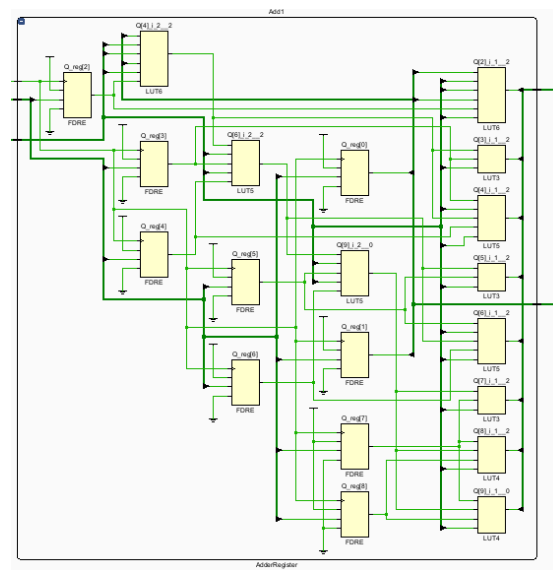


Figura 11: Particolare del modulo AdderRegister sintetizzato

Ricaviamo i dati sull'utilizzazione delle LUTs (Look-Up Table) e dei Flip-Flops impiegati nella realizzazione del nostro circuito nell'FPGA. Ricordiamo che, in questa fase, il software Vivado si limita a sintetizzare la logica behavioral del

nostro circuito senza implementarla all'interno del dispositivo reale. Per questa ragione, i dati ricavati sono delle stime di quelli che saranno i reali sull'utilizzazione che otterremo nella successiva fase di implementazione. Riportiamo di seguito i risultati ottenuti:

Listing 6: Costi di utilizzazione

Ref	Name	Used	Functional Category
FDRE		113	Flop & Latch
IBUF		49	IO
LUT5		22	LUT
LUT3		17	LUT
LUT6		13	LUT
OBUF		11	IO
LUT4		9	LUT
LUT2		8	LUT
BUFG		1	Clock

La prima voce della tabella riporta il numero di Flip-Flops impiegati, la seconda e la sesta rappresentano rispettivamente i buffer di input e di output, l'ultima il segnale di clock e tutte le altre il numero di LUTs utilizzate divise per il numero di ingressi che ammettono (indicato dal numero finale di ogni voce).

Nella fase di sintesi bisogna anche impostare un constraint (="vincolo") sul segnale di clock. Omettere questo passaggio significa lasciare libero il circuito di lavorare ad una qualunque frequenza, anche estremamente alta. Ciò comporta una dissipazione da parte del circuito di un'enorme quantità di potenza dinamica. Infatti sappiamo che vale la seguente:

$$P_{dinamica} = f \cdot C_L \cdot V_{DD}^2$$

dove f è la frequenza di lavoro del circuito, C_L è la capacità dei transistori del circuito e V_{DD} è la tensione di alimentazione. Quindi se la frequenza f è libera di aumentare senza il vincolo del constraint anche la potenza dinamica dissipata aumenta indefinitivamente. Impostiamo allora, usando il software Vivado, il periodo di clock a $20ns$ con duty cycle del 50%. Il software produrrà un constraint che traduce nel seguente comando:

Listing 7: Constraint del clock

```
create_clock -period 20.000 -name myClock
            -waveform {0.000 10.000} [get_nets clk]
```

3.1 Simulazione post-sintesi

Conclusa anche questa fase, andiamo a verificare la correttezza di quanto ottenuto svolgendo una simulazione temporale post-sintesi. Prima di fare questo, però, si decide di modificare l'attuale testbench nel seguente modo

Listing 8: File TestBench.vhdl

```

1  ...
2  -- ingressi di test
3  process begin
4      wait for 105 ns; -- modifica apportata
5      for i in -10 to 9 loop
6          A <= conv_std_logic_vector(i, 8);
7          E <= conv_std_logic_vector(127 - i, 8);
8          B <= conv_std_logic_vector(-128 + i, 8);
9          C <= conv_std_logic_vector(63 + i, 8);
10         D <= conv_std_logic_vector(-32 - i, 8);
11         F <= conv_std_logic_vector(-16 + i, 8);
12         wait until falling_edge(clk);
13     end loop;
14 end process;
15 ...

```

In particolare, è stato introdotto un tempo di attesa pari a 105 ns prima dell'applicazione dei primi segnali in ingresso alle varie porte. Questa scelta è motivata dalla presenza, all'interno degli FPGA, di un segnale di global reset attivo alto (GSR), che nei primi 100ns resetta il contenuto di tutti i registri presenti nel circuito, portandoli allo stato 0. Nel nostro caso, poiché il clock ha un periodo di 20ns con una duty cycle del 50%, la sua forma d'onda presenta un fronte di discesa proprio al tempo $t = 100ns$. Ciò comporta che la simulazione cambia immediatamente il primo gruppo di segnali in quanto la riga *wait until falling_edge(clk)* troverà la condizione immediatamente verificata. Proprio per questo abbiamo portato il tempo di attesa a 105ns. Si potrebbe pensare di variare gli ingressi in un istante temporale diverso dal fronte di discesa; tuttavia, questa scelta riduce la probabilità di violare i vincoli di setup o hold, che nel nostro contesto sono collocati nell'intorno del fronte di salita, e quindi di incorrere in fenomeni di metastabilità.

Di seguito è riportato l'esito della simulazione, ottenuto tramite il tool Vivado, nel quale si osserva una latenza pari allo stesso numero di cicli di clock già evidenziato nella simulazione precedente.

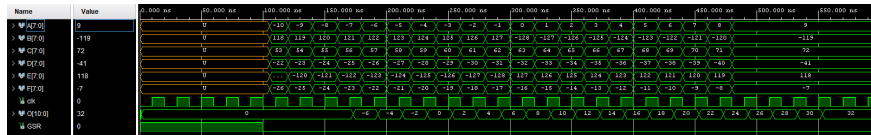


Figura 12: Simulazione temporale post sintesi

4 Implementazione

In questa fase si va ad implementare il circuito sintetizzato nel dispositivo hardware FPGA. Successivamente a questo, vengono estratti i dati definitivi sull'occupazione delle risorse (come LUTs e Flip-Flops) necessarie alla realizzazione fisica del circuito. Mentre i report della sintesi fornivano una stima approssimativa, i dati ricavati nell'implementazione indicano l'effettivo numero di risorse allocate e cablate all'interno dell'FPGA.

Listing 9: Costi di utilizzazione

Ref	Name	Used	Functional Category
FDRE		113	Flop & Latch
IBUF		49	IO
LUT5		22	LUT
LUT3		17	LUT
LUT6		13	LUT
OBUF		11	IO
LUT4		9	LUT
LUT2		8	LUT
BUFG		1	Clock

Come si evince dal report, i risultati attuali coincidono con quelli ottenuti in fase di sintesi. Questo conferma che le stime iniziali rappresentavano un'approssimazione accurata delle risorse effettivamente impiegate nell'implementazione fisica.

I vincoli sul clock impostati precedentemente vengono mantenuti anche per la fase di implementazione fisica. Quindi si ha una periodo di clock di 20 ns con un duty cycle del 50 %.

Poiché la fase corrente riguarda la realizzazione fisica del circuito, è possibile valutare con precisione la potenza dissipata. Nello specifico, verranno analizzate la potenza statica e la potenza dinamica, per determinare l'impatto energetico complessivo del design sul dispositivo. I valori forniti dal software sono:

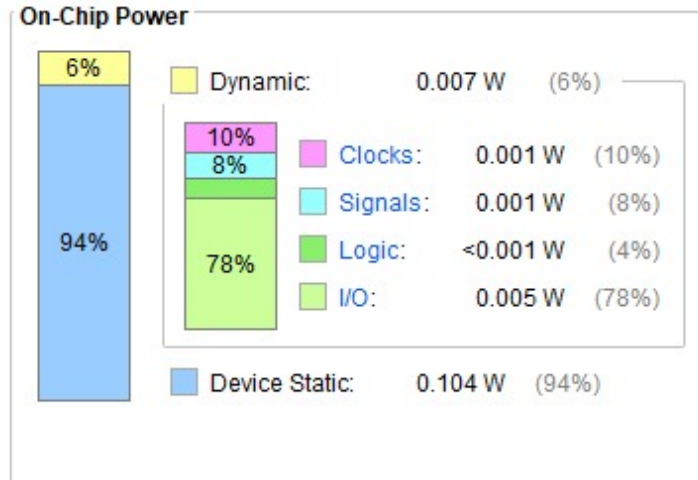


Figura 13: Report sulla potenza dissipata

Il report evidenzia come la potenza dinamica sia molto più piccola della statica. Ciò è dovuto principalmente alla bassa frequenza di lavoro derivante dal vincolo di clock di 20 ns . Poiché la potenza dinamica è direttamente proporzionale alla frequenza di commutazione, un periodo di clock così ampio limita la dissipazione energetica attiva del dispositivo.

Il software Vivado, in seguito all'implementazione del circuito, ci permette di visionare il Timing Summary Report in cui troviamo il **WNS** (*Worst Negative Slack*). Questo valore rappresenta il margine tra il ritardo effettivo del circuito e i requisiti di timing definiti dal periodo di clock. Nel nostro caso esso ha un valore pari a:

$$WNS = 16.718\text{ ns}$$

Il valore ottenuto è un valore positivo, ciò vuol dire che il nostro circuito può lavorare con un periodo di clock inferiore, così da aumentare la sua frequenza. D'altra parte, se questo valore fosse stato negativo, avrebbe significato che il periodo di clock indicato dal constraint sarebbe stato inferiore del ritardo combinatorio del circuito. In questo caso i registri, nell'istante del fronte di salita del clock, si sarebbero aggiornati con valori inconsistenti.

4.1 Simulazione post-implementazione

Realizzato il circuito reale, andiamo a effettuare una simulazione temporale post-implementazione:

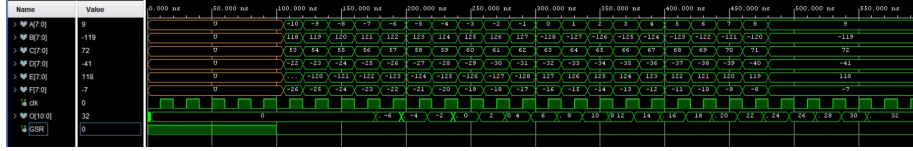


Figura 14: Simulazione post implementazione

Anche in questa simulazione è presente un tempo di attesa iniziale di 105 ns prima dell'applicazione dei segnali d'ingresso. Tale ritardo è stato impostato, come discusso in precedenza, per compensare l'azione del segnale di Global Set/Reset (GSR), che mantiene resettati tutti i registri del dispositivo per i primi 100 ns di operazione.

5 Massimizzazione della frequenza di funzionamento

Noto il *worst negative slack*, è possibile andare a lavorare sul constraint del clock al fine di massimizzare la frequenza di lavoro del componente. Come già accennato, l'aumento della frequenza comporta un'incremento di dissipazione di potenza dinamica. Il nuovo periodo da impiegare sarà calcolato come:

$$T' = T - WSN$$

con T l'attuale periodo e T' il nuovo periodo che si andrà ad impiegare. Da cui segue che la frequenza sarà pari ad

$$f' = \frac{1}{T - WSN}$$

Si presenta di seguito come è stato riscritto il file di constraint

Listing 10: Constraint del clock

```
create_clock -period 4.000 -name myClock
            -waveform {0.000 2.000} [get_nets clk]
```

Una volta effettuata tale modifica, si rende necessaria la riesecuzione delle fasi di sintesi e implementazione, dal momento che è noto come il fan-out influisca sui ritardi di propagazione e, di conseguenza, sulla velocità di funzionamento del circuito. Al fine di ridimensionare tale problematica, è possibile intervenire mediante tecniche di bufferizzazioni dei segnali, che nel nostro caso sono automaticamente svolte dal tool impiegato, le quali comportano un aumento dell'utilizzo delle LUT e delle altre risorse logiche impiegate (dunque una variazione della struttura sintetizzata ed implementata).

Per brevità non riportiamo un'attenta descrizione di tutti i passaggi compiuti

dal momento che sono del tutto analoghi a quelli visti nei precedenti paragrafi, però si riportano i risultati ottenuti dai vari report:

- **Potenza dinammica dissipata:** 0.036W

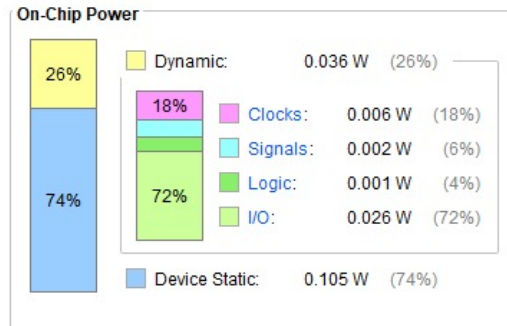


Figura 15: Report sulla potenza dopo aver aumentato la frequenza

Si osserva un incremento significativo rispetto al caso precedente.

- **Numero di componenti impiegati :**

Listing 11: Costi di utilizzazione

Ref Name	Used	Functional Category
FDRE	113	Flop & Latch
IBUF	49	IO
LUT5	22	LUT
LUT3	17	LUT
LUT6	13	LUT
OBUF	11	IO
LUT4	9	LUT
LUT2	8	LUT
BUFG	1	Clock

In questo caso non si notano variazioni; ciò potrebbe indicare che non siano avvenuti particolari processi di bufferizzazione.

- **Worst Negative Slack :** 1.073 ns, valore sensibilmente inferiore rispetto a quello precedente.

Dal momento che il WNS ottenuto non è né nullo né prossimo allo zero, si potrebbe pensare di aumentare ulteriormente il valore della frequenza di progetto, ma si è deciso di non percorrere questa strada per evitare ulteriori accrescimenti della potenza dinamica dissipata.